

HERITAGE SHIELD

SIH '26 · Team 031

AI-Assisted Digital Twin for
Predictive Conservation of
India's Cultural Heritage

Complete project, technology,
implementation, prototype,
scalability and evaluation brief

A concise technical brief for presentation, implementation and Round 2 execution.

1. Executive Summary

Heritage Shield is a predictive conservation and heritage-intelligence platform. It converts fragmented heritage evidence—historical photographs, inspection records, GIS/hazard information and new observations—into a living digital twin of an asset. The system tracks component-level condition over time, detects visible deterioration, estimates risk, predicts deterioration trends and ranks where conservation attention is needed first.

It is not a disaster-management app, virtual-tour application or static 3D archive. Disaster and environmental hazards are inputs to the risk layer. The core product is a decision-support system for preventing or reducing irreversible heritage loss.

Round 2.1 expects 30–60% progress, a 10–12 minute presentation/demo and 5–7 minute Q&A.; Round 2.2 requires 100% completion, stable end-to-end functionality, technical depth, scalability, feedback incorporation and strong Q&A.;

Core loop:



20-second pitch: Heritage Shield creates a living digital twin, detects deterioration, predicts what is likely to happen next and tells authorities what to prioritise before damage becomes irreversible.

2. Problem — Exactly What We Are Solving

Problem statement: Heritage sites are continuously exposed to structural deterioration, environmental stress, human activity and natural hazards. Existing condition assessment and conservation planning can depend on fragmented records and periodic inspections, making it difficult to detect deterioration trends early and prioritize preventive intervention before damage becomes irreversible.

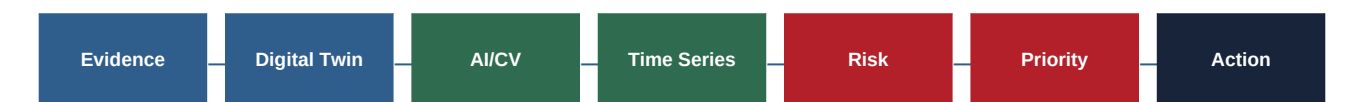
Root problem: data exists, but observations are not consistently connected to the same physical components and converted into a continuous, actionable decision record.

Layer	Current issue	Consequence
Data	Photos, reports, GIS and hazard information are fragmented	Evidence is difficult to combine
Inspection	Periodic snapshots rather than continuous history	Slow detection of worsening conditions
Analysis	Damage may be recorded without tracking progression	Rate of deterioration is unclear
Decision	Limited resources must be prioritised	Attention may not go first to the most urgent component

Users: heritage authorities, conservation architects/structural experts, site managers/field officers, local authorities and conservation teams with limited inspection resources.

3. Solution and Product Definition

Heritage Shield combines a dynamic heritage profile, component-level digital twin, AI visual assessment, temporal change history, risk engine and conservation prioritisation layer. The 3D model is the spatial interface; the real product is the decision workflow.



Module	What it does	Output
Heritage profile	Stores identity, materials, significance, inspections and history	Structured asset record
Digital twin	Represents site and important components in 2D/3D	Spatial component model
AI visual assessment	Flags cracks, dampness, vegetation, surface damage and other visible anomalies	Damage/anomaly candidates
Temporal change detection	Compares observations across dates	Change magnitude/trend
Health/Vulnerability index	Combines interpretable condition and exposure factors	Explainable score
Prediction	Uses historical condition + risk factors	Deterioration trend
Priority engine	Ranks components by urgency	Inspection/conservation queue
Recommendation	Turns evidence into a suggested action	Human-reviewed action

4. Technology Workflow — Feature by Feature

Feature	Technology	Use / implementation	Scale
Data ingestion	Python, FastAPI	Validate and normalise images, records, GIS/context; store metadata/files	Automated connectors; add sources without redesigning UI
3D reconstruction	Photogrammetry, COLMAP, Blender	Images → reconstruction → mesh/texture → optimised GLB/GLTF	Automated scan-to-twin; manual modelling only for exceptions
Interactive twin	Three.js, React/Next.js	Load model; map component IDs to database records	Reusable across many assets
Computer vision	Python, OpenCV, PyTorch/pretrained CV	Preprocess → inference → detection/segmentation → confidence → review	Batch cloud/GPU inference; retrain as labels grow
Temporal comparison	Image registration/comparison + time-series DB	Align observations → compare → calculate change features	Every new inspection becomes another time-series point
Risk engine	Python ML/analytics + explicit scoring	Condition + deterioration + hazard + importance → score	Weights/models recalibrated as evidence grows
GIS/hazard	PostGIS + Leaflet/Mapbox	Store geometry; spatial joins; render exposure/risk	Regional/national spatial queries
Prediction	Python ML/time-series	Condition history + environmental/hazard features → forecast + uncertainty	Improves with longitudinal data
Backend	FastAPI	APIs for assets, observations, inference, scores, priorities	Stateless services can scale horizontally
Database	PostgreSQL/PostGIS + object storage	Structured history in DB; images/models in object storage	Indexes/partitions + object storage support large asset counts
Frontend	Next.js/React + Three.js	Dashboard, map, twin, charts and review UI	Reusable components
Automation	Background jobs/queues + cloud	Ingestion → reconstruction → assessment → comparison → scoring → priority	Critical for 100s/1000s; removes manual repetition

5. End-to-End System Architecture

INPUTS	PROCESSING	INTELLIGENCE	STORAGE / OUTPUT
Historical photographs Inspection records New images GIS/hazard Environmental data			
Validation Pre-processing 3D reconstruction Image alignment Feature extraction			
CV assessment Change detection Health/risk score Prediction Priority generation			
PostgreSQL/PostGIS Object storage Twin assets FastAPI Next.js/Three.js dashboard			

Runtime path



Engineering separation: frontend = presentation; FastAPI = API/orchestration; ML/CV workers = inference; PostgreSQL/PostGIS = structured state; object storage = large files; background jobs = expensive processing.

Source of truth: scores, detections, histories and priorities must be generated/stored by backend services and database records, not hard-coded in the UI.

6. Database / Data Model

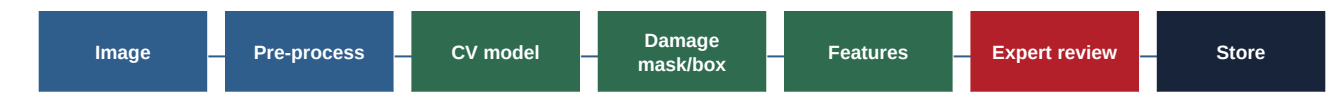
Entity	Key fields	Purpose
HeritageAsset	asset_id, name, location, significance, material, period	One asset → many components, inspections and observations
Component	component_id, asset_id, type, geometry, importance	Unit of condition tracking and prioritisation
Inspection	inspection_id, asset_id, date, source	Groups observations from an inspection cycle
Observation	observation_id, component_id, inspection_id, image_uri, condition features	Time-series evidence
Detection	detection_id, observation_id, defect_type, confidence, geometry	CV output requiring review
RiskAssessment	component_id, date, factors, score, uncertainty	Explainable health/risk state
Prediction	component_id, horizon, predicted_value, uncertainty	Future trajectory
HazardExposure	component/asset, hazard_type, severity, source	Flood/seismic/fire/extreme-weather/environmental context
Intervention	component_id, action, urgency, status, outcome	Recommendation and conservation outcome

Large-file rule: store images, point clouds, meshes and model files in object storage; keep metadata, paths, geometry and relationships in PostgreSQL/PostGIS.

7. CV/ML Architecture and Risk Computation

Round 1 feedback to address: deeper root-cause diagnosis, technical/model-level specificity, measurable validation, rigorous ground truth, multimodal pipelines and explicit CV/ML + risk-score computation.

CV pipeline



Early defect classes: cracks, dampness/moisture indicators, vegetation intrusion, surface loss/discoloration and other visible anomalies. Model outputs are review suggestions, not engineering verdicts.

Temporal pipeline



Interpretable risk formula: Risk Score = $w_1 \cdot \text{Condition} + w_2 \cdot \text{Deterioration Rate} + w_3 \cdot \text{Hazard Exposure} + w_4 \cdot \text{Heritage/Structural Importance}$.

The dashboard should expose the factors driving the score. Candidate dimensions include structural condition, material condition, environmental stress, moisture risk, human pressure, climate/hazard exposure and historical significance.

Prediction: use longitudinal component observations plus environmental/hazard features to estimate a trajectory; show uncertainty bands rather than false precision.

8. Validation — What Must Be Measured

Layer	Metric / KPI	Purpose	Evidence
Damage detection	Precision, Recall, F1, IoU where applicable	CV usefulness	Expert-labelled test images
Temporal change	Change error / agreement with expert labels	Deterioration quantification	Paired historical/current observations
Prediction	MAE/RMSE + calibration/uncertainty	Forecast quality	Held-out time periods
Risk ranking	Agreement with expert priority ranking	Useful prioritisation	Expert assessment / field records
Operations	Processing time, assets/batch, flag→review time	Practicality	System logs
Conservation workflow	Inspection effort, turnaround, priority acceptance	Operational impact	Pilot records

Ground truth: combine historical/current imagery, inspection reports and expert-labelled condition observations. Separate evaluation by site/time where possible. High-fidelity structural claims require domain-appropriate ground truth.

9. Phase-by-Phase Building Process

Phase	Build	Exit criterion
0. Scope/data	Choose 1–2 sites; define components, defects, inputs and outputs	One complete use case is fixed
1. Data foundation	Collect/organise images, records, GIS/hazard; define schema	Every observation has asset/component/date/source
2. Digital twin	Generate/import 3D model; map components to IDs; optimise GLB/GLTF	Selecting a component returns its record
3. Backend	FastAPI + PostgreSQL/PostGIS + object storage	Frontend can use real API data
4. CV	Baseline detector + expert review + stored detections	Known test images produce measurable outputs
5. Temporal	Align repeated observations; compute change features	Same component compares across dates
6. Risk	Explicit score + factor breakdown + hazard joins	Inputs change → score changes
7. Prediction	Baseline time-series/ML + uncertainty	Forecast evaluated on held-out data
8. Priority	Rank components + inspection/action suggestions	Authority sees what to inspect first and why
9. Integration	Connect twin, assessment, comparison, health, risk, priority and dashboard	One end-to-end path runs
10. Automation	Background jobs, retries, logs, batch processing	New asset does not require manual UI work
11. Validation	Metrics, error analysis, realistic data, tests, deployment	Stable final demo with evidence

10. Entire Product Wireframe — First Step to Last

Step	Screen / user action	System response
1	Landing / authority dashboard	Assets, risk distribution, alerts and inspections
2	India / regional map	Heritage assets with risk status
3	Select site	Heritage profile + digital twin
4	Select component	Component geometry, history and inspection trail
5	Upload / ingest observation	Validate and store image/record/context
6	AI assessment	CV flags anomalies with confidence
7	Temporal comparison	Historical/current comparison and change
8	Health / risk	Score + factor contributions + hazard exposure
9	Prediction	Deterioration trajectory + uncertainty
10	Scenario/hazard	Apply supported hazard/environment condition
11	Priority queue	Rank components/sites by urgency
12	Recommendation	Suggested action and timing
13	Expert review	Accept/edit/reject recommendation
14	Intervention record	Log action and outcome
15	Learn/update	New evidence enters component history

Prototype navigation already follows: Digital Twin → Assessment → Change Detection → Health Index → Risk Layer → Priority Queue → Dashboard.

11. Prototype Status — What Exists vs What Is Next

Module	Current prototype	Engineering status
Dashboard	4 demo sites; average health index; urgency; inspection cycle	UI prototype / demo data
Digital twin	Site selector, hotspots, component detail	Connect to generated/real 3D + DB
AI assessment	Interactive mock findings for crack/dampness/vegetation	Replace mock findings with actual CV
Temporal comparison	Interactive slider	Connect to stored images + change algorithm
Health index	Factor breakdown and explanation	Formalise backend formula + calibration
Risk map	Site markers	Connect to real spatial/hazard datasets
Prediction	Observed + predicted trajectory + uncertainty band	Connect to validated model
Priority queue	Ranked components with score/status/action	Connect to backend risk engine
Recommendation	Problem → factors → action → human sign-off	Connect to intervention records
Authority dashboard	Trend, inspections, risk distribution, intervention history	Connect to live DB

Honesty rule: the current HTML prototype contains mock/static demonstration values and interactions in several modules. Do not present mock CV findings or mock scores as validated AI results. Replace the critical path with real/realistic data and working backend outputs for the next milestone.

12. Round 1 Feedback → Required Action

Feedback	Action
Deeper root-cause diagnosis	Explicitly show fragmentation → temporal tracking → decision priority.
Technical/model specificity	Show CV pipeline, model inputs/outputs and backend architecture.
Elaborate CV/ML architecture	Name processing stages and evaluation metrics; separate baseline from future.
Risk-score computation	Use explicit factors; expose contributors; document/calibrate weights.
Measurable validation evidence	Add ML metrics and operational KPIs.
Rigorous ground truth	Use expert-labelled/inspection-backed observations; avoid structural-certification claims.
Multimodal pipeline	Connect imagery + inspection + GIS/hazard + environmental data.
Realistic, implementable, scalable	Focused MVP, specific remaining work, automation-based scale path

Status labels to use everywhere: IMPLEMENTED / IN DEVELOPMENT / PLANNED / SCALE-UP. This prevents overclaiming and improves credibility.

13. Round 2 Rubric — What Must Be Proven

Criterion	Marks	Proof needed
Working Prototype / Live Demo	10	Real functioning modules; stable demo
Technical Implementation & Architecture	10	Justified tech, architecture, data flow and code design
Progress vs Roadmap / Alignment	10	Planned vs completed scope; no drift
Feasibility of Remaining Work	5	Specific tasks, risks and mitigation
Handling Round 1 Feedback	5	Visible CV/ML, risk, validation, ground-truth changes
Q&A;	10	Every member can defend technical decisions

Round 2.1: 30–60% expected progress; 10–12 min presentation/demo + 5–7 min Q&A.; **Round 2.2:** 100% completion; 10–12 min demo/presentation + 5–8 min Q&A; real/realistic data; end-to-end functionality.

14. Scalability — 1 Site to 1000s

Scale	Engineering change	Core product stays
1–2 sites	Controlled dataset; focused model; simplified twin	Asset/component/evidence/risk workflow
10–50	Automated ingestion; more labels; batch inference; GIS expansion	Same APIs and UI concepts
100s	Cloud queues/workers; object storage; model registry; monitoring	Same IDs/history/risk/priority logic
1000s+	Automated scan-to-twin; distributed workers; scheduled reprocessing; region/state dashboards	Same component-centric decision layer

Critical scalability principle: never make manual 3D modelling the required path for every asset. Automate ingestion/reconstruction and reserve expert work for exceptions and review.

Future extensions: IoT sensors, drones, LiDAR, AR field inspection, climate forecasting, knowledge graph and verified citizen observations. These are extensions, not mandatory MVP dependencies.

15. Risks and Mitigation

Risk	Mitigation
Limited labelled data	Narrow initial defect classes; expert labels; transfer learning; held-out evaluation
Different materials/styles	Component/material metadata; site-aware evaluation; incremental retraining
3D quality	Capture guidelines; quality checks; simplified twin for MVP
CV false positives/negatives	Human review; confidence; audit trail; no structural certification
Risk-score subjectivity	Expose factors; document weights; calibrate against expert priorities
Sparse time series	Conservative baselines; uncertainty; update with each inspection
Large files/compute	Object storage; asynchronous workers; optimized assets; batching
Scope creep	Keep core loop fixed; treat extra technologies as extensions

16. Ideal Live Demo Sequence

Order	Show	Say
1	Authority dashboard	"This is the decision layer: what is urgent, where and why."
2	Select site	"We open the heritage asset and its digital profile."
3	Digital twin	"The twin maps condition to individual components."
4	Component	"Each component has its own inspection and condition history."
5	AI assessment	"CV flags visible anomalies for expert review."
6	Temporal comparison	"We compare observations across dates to quantify change."
7	Health/risk	"The score exposes the factors driving the result."
8	Prediction	"Historical condition and risk factors produce a trajectory with uncertainty."
9	Priority queue	"The system ranks where attention should go first."
10	Recommendation	"The system recommends; the conservation professional makes the final decision."

Demo rule: one complete path is stronger than many partially functional screens. If a module is mock, say so.

17. Recommended 10–12 Minute Presentation

Part	Time	Purpose
Problem + root cause	1:00–1:15	Exact problem and gap
Existing gap	0:40–0:50	Why current approaches do not close the workflow
Solution	0:50–1:00	Product loop
Technical architecture	1:10–1:20	Round 1 technical feedback
CV + risk	1:00–1:15	Model specificity + explainability
Live prototype	3:00–3:30	Evidence of working modules
Validation + roadmap	0:50–1:00	Success measures + remaining work
Feedback addressed	0:40–0:50	Visible Round 1 changes
Closing	0:15–0:20	Decision-support value

18. Q&A; — Questions Everyone Must Know

Why digital twin?

It provides persistent spatial/component identity so repeated observations can be connected over time.

Why AI?

For scalable visual assessment, change analysis and prediction; AI is a processing layer, not the product.

How will you validate?

Expert-labelled data + held-out tests + precision/recall/F1/IoU or forecasting metrics + operational KPIs.

How is risk calculated?

Interpretable factors: condition, deterioration rate, hazard exposure and importance; weights are calibrated.

What if AI is wrong?

Human review, confidence and audit trail; no autonomous structural-safety verdict.

Where does data come from?

Controlled historical/current imagery, inspection records and GIS/contextual data for MVP; expand through authorities/field capture.

How does it scale?

Automated ingestion, scan-to-twin processing, queues, object storage, spatial DB and batch inference.

What works now?

Only claim what the prototype/backend can actually demonstrate; label mock data clearly.

Biggest technical risk?

Ground-truth quality and generalisation; mitigate with narrow scope, expert labels and incremental validation.

19. Source Basis and Existing References

Source	Used for this brief
SIH26_031-Team_Qualified.pdf	Submitted problem, solution, tech stack, differentiation, feasibility and references.
Round2_Evaluation_Rubrics.pdf	Round 2.1/2.2 marks, time limits, progress expectations and evaluation focus.
Heritage_Shield_SIH_2026_Brief.pdf	Expanded product loop, data pipeline, health index, MVP, demo flow, architecture and scalability.
heritage-shield-prototype.html	Current prototype modules, UI workflow, demo values and behaviour.
VISIONCRAFT PPT	Reference for concise technical pipeline and feasibility/demo framing.
First-round guidelines	Original evaluation criteria, citation expectations and low-score failure modes.

References retained from the submitted deck:

[1] NMMA — Ministry of Culture, Government of India. Documentation and National Heritage Database.

[2] Jouan & Hallot (2020), “Digital Twin: Research Framework to Support Preventive Conservation Policies,” ISPRS International Journal of Geo-Information, 9(4), 228.

[3] Kong & Hucks (2023), “Preserving our heritage: A photogrammetry-based digital twin framework for monitoring deteriorations of historic structures,” Automation in Construction, 152, 104928.

[4] Mishra & Lourenço (2024), “Artificial intelligence-assisted visual inspection for cultural heritage: State-of-the-art review,” Journal of Cultural Heritage, 66, 536–550.

[5] Colace et al. (2026), “A computational approach to predictive conservation,” Digital Applications in Archaeology and Cultural Heritage, 40, e00519.

Reference hygiene: verify every bibliographic detail and official/DOI link against the source document before the final submission; do not add unverified references.

20. Immediate Next Steps — Priority Order

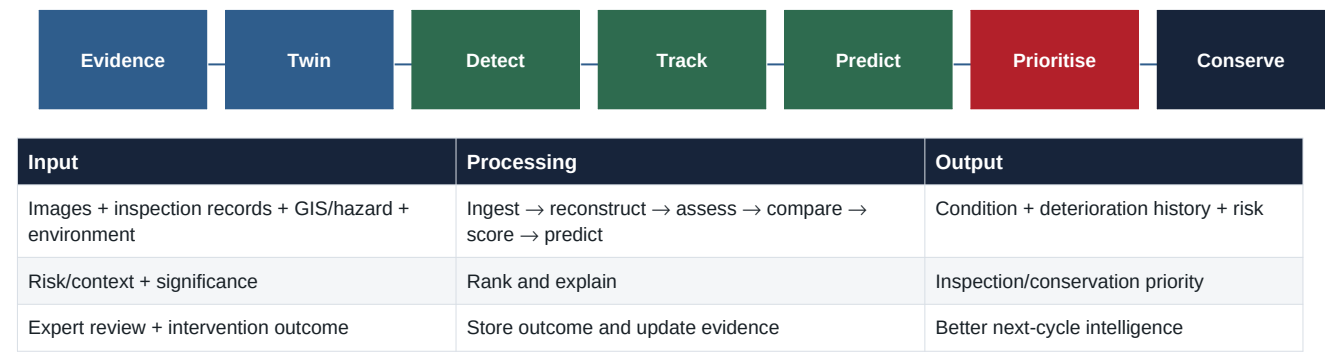
Priority	Action	Reason
P0	Lock one end-to-end component use case + real/realistic dataset	Makes the demo defensible
P0	Replace mock CV findings on the critical path with actual inference	Directly addresses prototype/technical marks
P0	Implement backend + DB for asset/component/inspection/observation	Turns UI into a real system
P0	Implement/document risk formula + factor breakdown	Direct evaluator feedback
P1	Build expert-labelled CV evaluation set	Enables measurable metrics
P1	Connect temporal comparison to stored observations	Makes change detection real
P1	Add defensible baseline prediction + uncertainty	Demonstrates prediction without overclaiming
P1	Connect GIS/hazard data to risk	Completes risk-aware layer
P2	Automate jobs + logs + retries	Shows true scalability
P2	Run end-to-end tests + demo backup	Protects live-demo score
P2	Prepare every member for Q&A;	10 marks now, 15 marks in final

Next-checkpoint definition of done: one real/realistic heritage asset → one component → observation → CV result → temporal change → explainable risk → prediction → ranked priority → human-reviewed recommendation, all connected through the actual backend and database.

21. The Entire Project in One Page

Problem: Heritage condition evidence is fragmented and inspections are periodic, so deterioration trends and priorities are difficult to track.

Solution: A living, component-level digital twin connected to AI/CV, temporal analysis, risk intelligence and conservation prioritisation.



Core technology: Next.js/React + Three.js; Python/FastAPI; PostgreSQL/PostGIS; object storage; OpenCV/PyTorch/CV models; COLMAP/photogrammetry + Blender; GIS/hazard layers; ML/time-series; automated background processing.

Core differentiation: not the 3D model alone. The value is connecting component identity, condition history, change detection, risk context and action priority in one workflow.

Scale: automate ingestion and processing so new assets follow the same pipeline; use cloud workers, object storage and spatial databases; reserve expert review for uncertain detections and final conservation decisions.

Credibility rule: show what works, measure what can be measured, label prototype/mock components honestly, and never claim engineering-grade prediction or structural safety without validated ground truth.

END OF BRIEF